Info 502 - TP 4

Université Savoie Mont Blanc - L3 INFO

Automne 2023

1 Consignes

Consignes usuelles

Pour chaque question, vous implémenterez des tests associés, dans une méthode appelée **question{n^o}** , permettant de tester votre code sur plusieurs exemples pertinents.

1.1 Installation guidée, qui fonctionne sur les PCs de l'université

La procédure ci-dessous fonctionne pour les ordinateurs sous Debian de l'université. Celle-ci a été testée et approuvée (dernière vérification le 12 juin 2023).
Pour tout autre ordinateur/toute autre configuration, voir ci-dessous.

Téléchargez les fichiers `info502-tp4.zip`, `info502-tp4.launch`

1.1.1 Installation d'Eclipse

1. Rendez-vous sur `https://www.eclipse.org/downloads/`
2. Choisissez *Download x86_ 64*
3. Désarchivez le fichier : `eclipse-inst-jre-linux64.tar.gz`
4. En ligne de commande, lancez le fichier `eclipse-inst` (dans le répertoire `eclipse-installer`)
5. Choisissez `Eclipse IDE for Java Developper`

1.1.2 Projet TP4

Choisissez : *File* → *Import...* → *General* → *Existing Projects into workspace*

Dans cette nouvelle fenêtre, choisissez *Select archive file*, et sélectionnez votre fichier `info502-tp4.zip`.

1.1.3 Cup configuration

Choisissez : *File* → *Import...* → *Run/Debug* → *Launch Configurations*

Dans cette nouvelle fenêtre, choisissez le **répertoire** où vous avez enregistré le fichier `info502-tp4.launch`, et sélectionnez votre fichier `info502-tp4.launch`.

1.1.4 Testez !

1. Choisissez : *Run* → *External tools* → *info502-tp4*
2. Choisissez : *Run* → *Run*

Si vous n'avez plus d'erreurs, vous pouvez lancer débuter le TP !

1.2 Autres Configurations

Cette procédure n'en est pas une. Il vous faudra compter sur votre connaissance de votre machine, votre perspicacité et votre persévérance ; ce dont vous ne manquez pas !

1.2.1 Installation du projet

Récupérez l'archive `info502-tp4.zip`, dézippez-là et ouvrez-là (avec Eclipse, voir ci-dessus, sinon à vous de jouer).

1.2.2 Cup configuration

En ligne, de commande, à la racine de votre projet, tapez l'équivalent de la commande suivante :

```
/usr/bin/java -jar jar/java-cup.jar -destdir src/fr/usmb/l3info/tp4/parser/
-package fr.usmb.l3info.tp4.parser -parser Parser src/cup/regex.cup
```

1.2.3 Testez !

Testez !

2 Les expressions régulières, données par une grammaire

On se souvient, que les expressions régulières sont données par les 6 cas suivants :

$$R \quad ::= \quad \emptyset \quad | \quad \varepsilon \quad | \quad s \quad | \quad R + R \quad | \quad R.R \quad | \quad R^* \qquad\qquad (s \in \Sigma)$$

C'est en fait une grammaire !

Dans ce TP, nous allons décrire cette grammaire, lever les ambiguités si il y en a, et vérifier qu'une expression régulière reconnait une chaine donnée.

Nous donnerons à notre programme une expression régulière du type $(a + b)a^*$, que l'on donnera à un *parseur* pour transformer cette chaine en un objet de type *RegEx*, que nous avons vu dans le TP3.

3 Mot vide : One

Prenez connaissance du fichier `regex.cup`. C'est ici qu'est définie notre grammaire. Actuellement notre grammaire ne reconnait pas les 6 éléments classiques.

Question 1 Ajoutez l'élément **ONE** à la grammaire.
Entre {: et :}, on peut ajouter du code, ici faites remonter la chaine de caractères *"1"* dans $RESULT$.
Décommentez la ligne correspondant dans le fichier *TP4Scanner*, et complétez la méthode *question1*.

4 Etoile de Kleene

Question 2 Ajoutez l'élément **STAR** à la grammaire.
Dans le fichier *regex.cup*, il faut penser à déclarer **STAR** comme élément terminal, et mettre à jour la variable $RESULT$.
Ajoutez une ligne dans le fichier *TP4Scanner*.

5 CONCAT, précédence

Question 3 Ajoutez l'élément **CONCAT** à la grammaire.

Il faut déclarer des expressions à gauche et à droite de **CONCAT**, comme suit :

```
    | expr:e1 CONCAT expr:e2      {: ..... :}
```

Question 4 Que se passe-t-il ?

La grammaire est ambigüe : en effet, $a.b.c$ peut être interprété comme $a.(b.c)$ ou bien comme $(a.b).c$. Si c'est indistinct pour nous, cela ne l'est pas pour notre parseur. Il faut préciser laquelle des deux possibilités on veut, en indiquant la précédence droite ou gauche (ie. qui a la priorité !) :

```
precedence <left/right> CONCAT;
```

6 PLUS

Question 5 Comment sont interprétées les expressions
$a + b^*.c$
$a.b^* + c$
Pour gérer les priorités des opérations +, . et *, il faut faire attention à l'ordre d'apparition des précédences.
Ajoutez une priorité pour l'étoile et n'oubliez pas de remplir la méthode *question5* pour vérifier que vous n'avez

pas mis les priorités à l'envers.

7 Concaténation implicite

On souhaite ajouter une règle pour permettre la concaténation implicite : on pourra ainsi écrire abc au lieu de $a.b.c$.

Question 6 Esssayer d'ajouter une règle pour la concaténation implicite. Que se passe-t-il ?

Comme la concaténation implicite ne correspond à aucun token, on ne peut pas déclarer de priorité ! Il va donc falloir désambiguer la grammaire à la main.

Question 7 Commentez votre grammaire pour le non-terminal *expr* (avec un bloc $/*...*/$ comme en Java), et recréez une nouvelle grammaire *expr* en la décomposant en catégories :

- **expr** continue d'exister et correspond aux regex arbitraires
- **term** correspond aux regex qui ne commencent pas explicitement par une somme :
  - $ab + cd*$ n'est pas reconnu par la grammaire term
  - $a(b + c).d$ est reconnu par la grammaire term, tout comme $a(bc*)*$ ou $(ab + cd*)$, où la somme est cachée par les parenthèses.
- **atom** correspond aux regex atomiques : 0, 1, les symboles, les expressions parenthésées, les étoiles, ...

Votre grammaire pour les expressions ressemblera donc à :

```
expr:
    ...
  | ...
  | ...

term:
    ...
  | ...
  | ...

atom:
    ...
  | ...
  | ...
```

Vérifiez que votre grammaire fonctionne correctement : $a + bc$, $ab + c$, abc^*, $a.b^*c + d$, $(ab + c)^*d + e$, ...

8 Chaines de caractères bien parenthésés

Question 8 Ecrire un programme qui permet de reconnaitre les chaines de caractères bien parenthésées. Vous aurez (entre autres) à écrire un fichier cup.

La grammaire bien parenthésée (qui n'aurait que des parenthèses comme caractères) est la suivante :

$$S \quad ::= \quad \varepsilon \quad | \quad (S)S$$